Advanced Day 8 — Session 8 (Hours 29–32)

Python Programming (Advanced) • Data-Driven App Integration and Checkpoint 4

Session 8 Overview

Topics Covered Today

- Hour 29 — Deeper SQL practice; ORM exposure as an optional extension
- Hour 30 — Plug the SQLite repository into the app
- Hour 31 — Search, filter, and pagination patterns
- Hour 32 — Checkpoint 4: data-driven app milestone

Day Goal

- Make SQLite the actual source of truth
- Add practical search backed by the repository
- Prove persistence end-to-end

Session 8 Outcomes

By the end of today, learners will be able to:

- Choose between ORM exposure and deeper SQL practice with clear tradeoff reasoning
- Swap storage backends without changing the service contract or UI
- Add a practical, safe search feature backed by parameterized SQL
- Prove persistence across restarts with a structured fast-grade demo
- Explain where each responsibility — model, service, repository, interface — lives

Scope Guardrails for Today

In Scope

- Deeper SQL summary queries with `GROUP BY` and `COUNT(*)`
- Optional tiny SQLAlchemy slice (one model, one session, one query)
- Storage swap keeping the service contract stable
- Parameterized `LIKE` search and `LIMIT` / `OFFSET` pagination
- Checkpoint 4 milestone: persistence, CRUD, search, and architecture review

Not Yet

- Migration frameworks or schema-generation tools
- Lazy loading, advanced ORM relationships, or session lifecycle deep dives
- Cursor-based pagination or dynamic search ranking

Hour 29 — Optional ORM Slice or Deeper SQL Practice

Learning Outcomes

- Explain in plain language what an ORM is and what problem it solves
- Compare ORM convenience vs raw SQL transparency without treating either as "the one right answer"
- Implement one practical extension to the SQLite-backed tracker app
- Describe the tradeoff between convenience and transparency when choosing a data-access layer

Default class path

- **Required for all cohorts** — deeper SQL practice

Why This Hour Exists

The architecture from Day 7 does the translation work manually:

- hand-written SQL statements
- manual row-to-object mapping
- explicit commits and connection management

Today's question — is there a tool that automates that mapping, and when does using it help vs hurt?

> "Abstraction is only helpful when it reduces confusion more than it adds confusion."
>
> — `Day8_Hour1_Advanced.md` → `## Section 5: Key Takeaway Script`

What an ORM Does

An ORM (Object-Relational Mapper) helps Python objects and relational tables speak to each other.

Typical ORM features:

- Python classes that correspond to tables
- Fields on those classes that correspond to columns
- A session or unit-of-work object for database interaction
- Query methods that generate SQL under the hood
- Declarative metadata close to the model definition

The cost: a new abstraction layer. If it is not understood, debugging becomes harder, not easier.

Raw SQL vs ORM — The Honest Comparison

Dimension	ORM	Raw SQL
Mapping boilerplate	Less	More
Dependency footprint	Heavier	Lightweight
Query transparency	Hidden	Visible
Portability	Framework-tied	Cross-language
Debug difficulty	Higher	Lower
Fit for small apps	Optional	Excellent

Key message — both are modern; both are useful. The right question is: "What helps this project and this team right now?"

Two Architecture Paths — Same Core Structure

```
Option A (ORM):
  UI/API  ->  Service  ->  ORM model/session  ->  SQLite

Option B (Raw SQL):
  UI/API  ->  Service  ->  SQL repository      ->  SQLite
```

What stays the same across both paths:

- the service layer
- the repository abstraction
- SQLite as the source of truth

The architectural continuity is what protects us from chaos.

Demo Track A — Tiny SQLAlchemy Slice

```python
from sqlalchemy import create_engine, Integer, String
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, Session

class Base(DeclarativeBase):
    pass

class RecordRow(Base):
    __tablename__ = "records"
    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    title: Mapped[str] = mapped_column(String)
    category: Mapped[str] = mapped_column(String)
    status: Mapped[str] = mapped_column(String)

engine = create_engine("sqlite:///data/tracker.db")
with Session(engine) as session:
    rows = session.query(RecordRow).all()
    for row in rows:
        print(row.id, row.title, row.status)
```

Demo Track B — SQL Summary Queries

```python
import sqlite3

def summarize_by_status(db_path: str) -> list[tuple[str, int]]:
    query = """
        SELECT status, COUNT(*) AS total
        FROM records
        GROUP BY status
        ORDER BY total DESC, status ASC
    """
    with sqlite3.connect(db_path) as connection:
        cursor = connection.execute(query)
        return cursor.fetchall()

for status, total in summarize_by_status("data/tracker.db"):
    print(f"{status}: {total}")
```

Why this path fits today — no extra package, transparent SQL, database does the counting.

Extending the SQL Path — Category Summary

```python
def summarize_category_for_status(
    db_path: str, status: str
) -> list[tuple[str, int]]:
    query = """
        SELECT category, COUNT(*) AS total
        FROM records
        WHERE status = ?
        GROUP BY category
        ORDER BY total DESC, category ASC
    """
    with sqlite3.connect(db_path) as connection:
        cursor = connection.execute(query, (status,))
        return cursor.fetchall()
```

Parameterized, practical, and answers a question a stakeholder might actually ask.

> Where should this live — repo, service, or a separate reporting helper?
> A bad answer is putting it inside a GUI callback.

Lab — Choose One Depth Path

Time: 25–35 minutes

Path B (required for all) — SQL Mini-Lab

1. Add one grouped summary query to the repository or reporting helper
2. Print a useful summary (e.g. records by status or by category)
3. Keep the query parameterized if it accepts user input
4. Add small output proving the query works

Path A (optional if environment allows) — ORM Mini-Lab

1. Install SQLAlchemy only if lab policy confirms package support
2. Define one model class matching the `records` table
3. Open a session, list rows, explain the mapping
4. Write one [illegible]

Common Pitfalls — Hour 29

- ⚠️ Installing a package without confirming environment support
- ⚠️ Switching architecture mid-course because the abstraction looked exciting
- ⚠️ Building SQL with string concatenation instead of parameters
- ⚠️ Putting summary queries directly in a GUI callback or Flask route
- ⚠️ Treating "more abstraction" as automatically more advanced

Quick Check

1. Name one advantage of an ORM.
2. Name one disadvantage of an ORM.
3. If you stayed with SQL, what new question can your database answer now?

Hour 30 — Integrate the Database into the App

Learning Outcomes

- Explain why the service layer should depend on behavior, not storage implementation
- Replace in-memory or JSON persistence with a SQLite-backed repository
- Identify what should and should not change when a storage backend is swapped
- Use dependency injection in a practical, beginner-friendly way
- Verify persistence across restarts without creating two conflicting sources of truth

The One Big Idea

> The service layer should depend on what the repository **can do**, not on **how** it does it.

The service should be able to say:

- add a record
- list records
- get a record by id
- update a record
- delete a record
- search records

Whether the repository stores data in a list, a JSON file, or a SQLite table is a **separate concern.**

What Stays Stable, What Changes

Stays the same

- Model classes
- Service method names and signatures
- Validation logic and exceptions
- UI callbacks and API routes

Changes freely

- Repository implementation details
- Connection setup and management
- SQL statements and row mapping
- Initialization steps like `init_db()`

Dependency Injection — The Right Pattern

```python
# Preferred: inject the repo so the service stays swappable
repo = SQLiteTrackerRepository("data/tracker.db")
service = TrackerService(repo=repo)
```

```python
# Avoid: service creates its own repo — harder to test and swap
class TrackerService:
    def __init__(self):
        self.repo = SQLiteTrackerRepository("data/tracker.db")
```

Benefits of the preferred pattern:

- Easier to test with a stub or in-memory repo
- Easier to switch persistence backends
- The wiring choice is visible at the entry point

Recommended Project Layout

```
src/
  tracker/
    models.py
    exceptions.py
    services.py
    repositories/
      base.py
      json_repo.py
      sqlite_repo.py
    db.py
gui/
  app.py
data/
  tracker.db
tests/
```

Folders do not need to match exactly. **Responsibilities must.** The service imports the repository contract; the GUI or API builds the service with the repository instance.

SQLiteTrackerRepository — init_db and add

```python
class SQLiteTrackerRepository:
    def __init__(self, db_path: str):
        self.db_path = db_path

    def init_db(self) -> None:
        query = """
            CREATE TABLE IF NOT EXISTS records (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                title TEXT NOT NULL,
                category TEXT NOT NULL,
                status TEXT NOT NULL CHECK (status IN ('open', 'done')),
                priority INTEGER NOT NULL DEFAULT 3,
                created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
            )
        """
        with closing(sqlite3.connect(self.db_path)) as conn:
            conn.execute(query)
            conn.commit()

    def add(self, record: Record) -> Record:
        query = "INSERT INTO records (title, category, status) VALUES (?, ?, ?)"
        with closing(sqlite3.connect(self.db_path)) as conn:
            cursor = conn.execute(query, (record.title, record.category, record.status))
            conn.commit()
            record.id = cursor.lastrowid
        return record
```

SQLiteTrackerRepository — list_all and wiring

```
    def list_all(self) -> list[Record]:
        query = "SELECT id, title, category, status FROM records ORDER BY id ASC"
        with closing(sqlite3.connect(self.db_path)) as conn:
            conn.row_factory = sqlite3.Row
            rows = conn.execute(query).fetchall()
        return [Record.from_row(row) for row in rows]
```

```
# Application entry point
def build_service() -> TrackerService:
    repo = SQLiteTrackerRepository("data/tracker.db")
    repo.init_db()
    return TrackerService(repo=repo)
```

The exciting part is not that SQLite exists. The exciting part is that the UI barely changed.

One Source of Truth

Once SQLite is live

- SQLite is the **only** live persistence layer
- JSON can become export/import, never a parallel truth

Danger pattern to eliminate

```
GUI  --writes-->  JSON file
Search  --reads-->  SQLite
Nobody knows which state is current
```

Say this explicitly to the class — "Once SQLite becomes your source of truth, stop treating JSON as live storage."

Canonical quiz cue — `Hour 30 | repository plugged in: yes`

Demo — Storage Swap With Minimal Drama

Demo flow

1. Build the SQLite repository
2. Call `repo.init_db()` at startup
3. Construct the service with that repo
4. Launch the app and confirm add/list/update/delete still work
5. Close the app completely
6. Reopen and verify the record still exists

Common integration bugs to watch for

- Method mismatch (`list_all()` vs `list_records()`)
- Forgetting `commit()` after inserts, updates, or deletes
- Mapping rows into dicts when the service expects model objects

Lab — Wire the Repository Under the Service

Time: 25–35 minutes

Required tasks

1. Confirm the SQLite repository implements all methods the service expects
2. Add `init_db()` to the startup entry point
3. Update the entry point to construct the service with the SQLite repository
4. Remove or disable the old JSON-as-primary-storage workflow
5. Prove add, list, update, and delete still work
6. Prove persistence survives a full restart

Completion Criteria

✓ App persists via SQLite
✓ UI/API still functions unchanged

Common Pitfalls — Hour 30

⚠️ UI calling `sqlite3` directly instead of going through the service
⚠️ JSON and SQLite both acting as "live" storage simultaneously
⚠️ Forgetting to refresh the UI list after database writes
⚠️ Rebuilding every class when only the repository implementation should change
⚠️ Pointing the GUI at one database file and tests at another

Quick Check

1. What interface stayed the same when you swapped storage?
2. Why is it risky to keep JSON and SQLite as simultaneous live sources of truth?
3. What part of the integration felt most fragile?

Hour 31 — Search, Filter, and Pagination Patterns

Learning Outcomes

- Add a practical search feature to a SQLite-backed application
- Use parameterized `LIKE` queries safely
- Explain the purpose of `LIMIT` and `OFFSET` in basic pagination
- Keep search and pagination logic in the repository layer
- Recognize common usability and correctness issues when data sets grow

Why Search Matters

An app stops feeling like a toy when users can find what they need.

- A task tracker with 300 records cannot just display everything
- Users want the app to answer a question quickly and safely
- Search and pagination are not glamorous, but they create trust

The design lens for this hour

> A user wants all items in "billing" that are still "open."
> They do not want to scroll forever.

Search is partly a UI feature, but the real correctness lives in the repository.

Search vs Filter — Know the Difference

Pattern	Meaning	Example
Search	Partial text match	title contains `invoice`
Filter	Exact or constrained value	status equals `open`

Clean combined repository method signature

```
def search(
    self,
    term: str = "",
    status: str | None = None,
    category: str | None = None,
    limit: int = 20,
    offset: int = 0,
) -> list[Record]:
    ...
```

Safe LIKE — The Right and Wrong Way

```
# WRONG: string concatenation — never do this
query = f"SELECT * FROM records WHERE title LIKE '%{term}%'"
```

```
# CORRECT: wildcard in the parameter value, not the SQL structure
wildcard = f"%{search_term.strip().lower()}%"
query = """
    SELECT id, title, category, status, priority, created_at
    FROM records
    WHERE lower(title) LIKE ?
       OR lower(category) LIKE ?
    ORDER BY id ASC
"""
rows = connection.execute(query, (wildcard, wildcard)).fetchall()
```

Two rules to remember:

1. Add `%` to the **parameter value**, never into the SQL string

LIMIT and OFFSET — Practical Pagination

```
SELECT id, title, category, status, priority, created_at
FROM records
ORDER BY id ASC
LIMIT 20 OFFSET 40;
```

- `LIMIT 20` — give me 20 rows
- `OFFSET 40` — skip the first 40 rows (i.e. show page 3 of 20)

What "practical" means in this course:

- one search box, maybe one status filter
- page size of 10 or 20
- optional Next/Prev controls

Critical rule — always add `ORDER BY` before `LIMIT` / `OFFSET`. Pagination without deterministic ordering creates duplicates and missing rows.

Full Repository Search Method

```python
def search(self, term: str = "", status: str | None = None,
           category: str | None = None,
           limit: int = 20, offset: int = 0) -> list[Record]:
    conditions: list[str] = []
    parameters: list[object] = []
    clean_term = term.strip().lower()
    if clean_term:
        wildcard = f"%{clean_term}%"
        conditions.append("(lower(title) LIKE ? OR lower(category) LIKE ?)")
        parameters.extend([wildcard, wildcard])
    if status:
        conditions.append("status = ?")
        parameters.append(status)
    if category:
        conditions.append("category = ?")
        parameters.append(category)
    where_clause = ("WHERE " + " AND ".join(conditions)) if conditions else ""
    query = f"""
        SELECT id, title, category, status, priority, created_at
        FROM records {where_clause}
        ORDER BY id ASC LIMIT ? OFFSET ?
    """
    parameters.extend([limit, offset])
    with closing(sqlite3.connect(self.db_path)) as conn:
        conn.row_factory = sqlite3.Row
        rows = conn.execute(query, tuple(parameters)).fetchall()
    return [Record.from_row(row) for row in rows]
```

Optional UI Hook — Page State in the Interface

```
def on_search(self):
    term = self.search_var.get()
    status = self.status_var.get() or None
    self.current_offset = 0   # always reset on new search
    records = self.service.search_records(
        term=term, status=status, limit=20, offset=0
    )
    self.populate_table(records)

def on_next_page(self):
    self.current_offset += 20
    records = self.service.search_records(
        term=self.search_var.get(),
        status=self.status_var.get() or None,
        limit=20,
        offset=self.current_offset,
    )
    self.populate_table(records)
```

Lab — Practical Search Feature

Time: 25–35 minutes

Recommended sequence

1. Start in the repo with a hard-coded search term in a small test script
2. Confirm the query returns expected rows
3. Move the method into the service layer
4. Connect the search term from the GUI or script input
5. Add pagination only after search is correct

Completion Criteria

✓ Repository search method uses parameterized SQL
✓ Search returns correct results
✓ Pagination uses stable `ORDER BY`

Common Pitfalls — Hour 31

⚠️ Putting SQL in the button callback instead of the repository
⚠️ Building SQL with string concatenation from user input
⚠️ Forgetting wildcards — `%` must be in the parameter value
⚠️ Paginating unsorted results — produces duplicates and missing rows
⚠️ Offset increasing forever without reset when a new search starts
⚠️ UI not clearing stale rows before displaying new results

Quick Check

1. Where should `%` be added — in the SQL string or the parameter value?
2. Why should paginated queries always have a stable `ORDER BY`?
3. What should the interface show when a search returns zero rows?

Hour 32 — Checkpoint 4: Data-Driven App Milestone

Learning Outcomes

- Demonstrate a SQLite-backed app that persists data across restarts
- Show clean CRUD behavior using the database as the true source of truth
- Explain where search logic and initialization logic live in the project
- Identify and fix common milestone bugs: schema mismatches and incorrect IDs
- Evaluate your own work against a checkpoint rubric focused on correctness, architecture, persistence, and professional habits

What This Hour Is — and Isn't

This is a proving hour, not a new-feature hour.

We are validating that the architecture from the last several sessions can survive contact with reality.

Instructor is looking for four things

1. SQLite is truly the source of truth
2. CRUD operations work reliably
3. Search works at a practical level
4. Code organization still makes sense under pressure

> "If your app is a little plain but stable, that is better than a flashy app that crashes. Stability is the priority."
>
> — `Day8_Hour4_Advanced.md` → `## Section 1: Framing the Milestone`

Required Deliverables — Checkpoint 4

Deliverable	Description
SQLite repository	Full CRUD — create, read/list, update, delete
Database initialization	`init_db()` runs on startup; safe to repeat
Source of truth	App reads and writes exclusively through SQLite
Basic search	Parameterized LIKE, returns correct results
Persistence proof	Record survives full close-and-reopen cycle

Stretch goal (not required) — a second table or join, safely separated from the core architecture.

Canonical quiz cue — `Hour 32 | db path: data/tracker.db`

Checkpoint Rubric — Plain Language

Level	Meaning
Meets	Core CRUD works; restart persistence proved; search works; SQL is parameterized; learner can explain the architecture
Approaching	Some workflows work but restart proof, search, or ID handling is unreliable
Needs revision	App still relies on in-memory or JSON state as live storage, or crashes during core workflows
Exceeds	Stable core milestone plus a clear smoke test or demo script; any extension safely separated

Correctness · Architecture · Error handling · Persistence · Professional habits

Fast-Grade Demo Workflow

This short sequence proves most of the milestone:

1. Launch the app — confirm `init_db()` runs without manual setup
2. Create a record — note the database ID
3. List records — confirm the new record appears
4. Search for the record by keyword
5. Close the app completely
6. Reopen the app
7. Confirm the record survived restart
8. Update the record by database ID
9. Delete the record by database ID
10. Search again — confirm the deleted record is gone

Validation Checklist

```
[ ] Database file exists in the expected path
[ ] init_db runs safely on startup (idempotent)
[ ] Add creates a new row in SQLite
[ ] List reads from SQLite, not stale in-memory data
[ ] Search returns expected results
[ ] Update targets the correct record ID
[ ] Delete removes the intended record
[ ] Restart proves persistence
[ ] Common errors do not crash the app
```

Work in this order — init → create/list → search → update/delete → restart → error handling

Smoke Test Script — Verify Before the Demo

```python
from pathlib import Path

def run_checkpoint_smoke_test() -> None:
    repo = SQLiteRecordRepository(Path("data/checkpoint_tracker.db"))
    repo.init_db()
    service = RecordService(repo=repo)

    created = service.add_record("Checkpoint persistence proof", category="demo")
    print(f"Created: {created.record_id} {created.title}")

    matches = service.search_records("persistence", limit=10, offset=0)
    print(f"Search matches after create: {len(matches)}")

    updated = service.mark_done(created.record_id)
    print(f"Updated: {updated.record_id} {updated.status}")

    service.delete_record(created.record_id)
    after_delete = service.search_records("persistence", limit=10, offset=0)
    print(f"Search matches after delete: {len(after_delete)}")
```

Common Failure Modes — How to Triage

Data disappears after restart

- No `commit()` after write operations
- Writing to one DB path and reading from another
- UI still rendering cached Python objects instead of re-querying

Wrong record changes on update or delete

- Using list index instead of stable database ID
- Wrong `WHERE` clause
- Stale selected ID in the UI state

Search returns strange results

- Wildcard placement errors (`%` in SQL, not parameter)

Professional Habits — Commit as You Go

When something significant works, commit it immediately:

```
feat: wire sqlite repo into tracker service
fix: use stable record id for update and delete
feat: add search query to sqlite repository
fix: reset pagination offset on new search
chore: remove json as parallel persistence path
```

Small commits make it easier to recover from mistakes.

Mini-architecture checks during build time

- Point to the repository in your project
- Explain where validation lives
- Explain how restart persistence is proved
- Explain why JSON is no longer the live source of truth

Showcase and Reflection

Demo format (keep each showcase under 2 minutes)

1. Show the app running from a clean launch
2. Create a record
3. Search for it
4. Restart and prove persistence
5. Explain one design decision

Reflection prompts

- What bug cost you the most time today?
- What design choice saved you time today?
- What feels stable now that did not feel stable yesterday?
- If a teammate joined your project tonight, what would confuse them first?

Session 8 Summary and What's Next

What Success Looks Like Today

- SQLite is now the real and only source of truth
- The storage swap preserved the service contract and the UI
- Search is practical, safe, and parameterized
- The checkpoint demo is honest, repeatable, and architecture-backed

Scope Guardrails — Held All Session

✓ Stable milestone before novelty
✓ Practical search and persistence over extra features
✓ ORM treated as optional exposure, not mandatory scope
✓ Clear repo / service / interface responsibility boundaries